

Database (SQL/MySQL) Practicals with Answers

[DDL](#) | [DML](#) | [DQL](#) | [Joins](#) | [Aggregates](#)

DDL: CREATE, ALTER, DROP · DML: INSERT, UPDATE, DELETE · DQL: SELECT, Filtering & Sorting · Aggregate Functions & GROUP BY · JOINS & Subqueries

Practical 1: DDL: CREATE, ALTER, DROP

Aim: Use Data Definition Language to create, modify and delete database structures.

Code:

```
-- Create database and use it
CREATE DATABASE college_db;
USE college_db;

-- Create table with constraints
CREATE TABLE students (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    branch VARCHAR(50) DEFAULT 'IT',
    marks DECIMAL(5,2) CHECK(marks >= 0 AND marks <= 100),
    enrolled_on DATE DEFAULT (CURRENT_DATE)
);

-- Create related table (Foreign Key)
CREATE TABLE placements (
    placement_id INT PRIMARY KEY AUTO_INCREMENT,
    student_id INT NOT NULL,
    company VARCHAR(100),
    package_lpa DECIMAL(5,2),
    placed_date DATE,
    FOREIGN KEY (student_id) REFERENCES students(student_id)
    ON DELETE CASCADE
);

-- ALTER: add column
ALTER TABLE students ADD COLUMN phone VARCHAR(15);

-- ALTER: modify column type
ALTER TABLE students MODIFY COLUMN phone VARCHAR(20);

-- ALTER: rename column (MySQL 8+)
ALTER TABLE students RENAME COLUMN phone TO contact;

-- ALTER: drop column
ALTER TABLE students DROP COLUMN contact;

-- Create index for faster queries
CREATE INDEX idx_branch ON students(branch);

-- DROP table
-- DROP TABLE placements;
-- DROP DATABASE college_db;
```

Explanation:

DDL commands define database structure. **CREATE TABLE**: defines columns with types and constraints. **PRIMARY KEY**: unique, not null identifier. **AUTO_INCREMENT**: auto-numbers rows. **NOT NULL**: mandatory field. **UNIQUE**: no duplicates. **DEFAULT**: value when not provided. **CHECK**: value validation. **FOREIGN KEY ... REFERENCES**: links tables. **ON DELETE CASCADE**: deletes child rows when parent deleted. **ALTER TABLE**: modify existing structure.

Output: Database and two tables created. students has 5 constraints. placements has FK to students with CASCADE. ALTER adds then removes a phone column. Index created on branch.

Practical 2: DML: INSERT, UPDATE, DELETE

Aim: Use Data Manipulation Language to insert, update and delete table records.

Code:

```
USE college_db;

-- INSERT single row
INSERT INTO students (name, email, branch, marks)
VALUES ('Yug Patel', 'yug@gtu.edu', 'IT', 88.5);

-- INSERT multiple rows at once
```

```

INSERT INTO students (name, email, branch, marks) VALUES
    ('Alice Shah', 'alice@gtu.edu', 'CE', 92.0),
    ('Bob Mehta', 'bob@gtu.edu', 'ME', 74.5),
    ('Carol Joshi', 'carol@gtu.edu', 'CE', 95.0),
    ('David Rana', 'david@gtu.edu', 'IT', 65.0);

-- INSERT placement records
INSERT INTO placements (student_id, company, package_lpa, placed_date)
VALUES
    (1, 'TCS', 4.5, '2024-11-01'),
    (2, 'Infosys', 5.0, '2024-11-15'),
    (4, 'Wipro', 6.2, '2024-12-01');

-- UPDATE single record
UPDATE students
SET marks = 90.0
WHERE student_id = 1;

-- UPDATE multiple columns
UPDATE students
SET branch = 'CSE', marks = marks + 5
WHERE branch = 'CE' AND marks < 95;

-- Conditional UPDATE with CASE
UPDATE students
SET branch = CASE
    WHEN branch = 'ME' THEN 'MECH'
    WHEN branch = 'CE' THEN 'CSE'
    ELSE branch
END;

-- DELETE specific row
DELETE FROM students WHERE student_id = 5;

-- DELETE with condition
DELETE FROM placements
WHERE placed_date < '2024-11-10';

-- TRUNCATE (delete all rows, keep structure)
-- TRUNCATE TABLE temp_table;

```

Explanation:

INSERT INTO table (cols) VALUES (...): add row. Omitting auto_increment column lets MySQL assign it. **UPDATE table SET col=val WHERE condition:** modify rows. Always use WHERE or all rows are updated. **CASE WHEN...END:** conditional update. **DELETE FROM table WHERE condition:** remove specific rows. **TRUNCATE:** faster than DELETE for all rows but can't be rolled back. Always test UPDATE/DELETE with SELECT first.

Output: 5 students inserted. 3 placement records added. Yug's marks updated to 90. Branch names changed via CASE. Student 5 deleted. Old placement removed.

Practical 3: DQL: SELECT, Filtering & Sorting

Aim: Query data using SELECT with WHERE, ORDER BY, LIMIT, LIKE, BETWEEN, IN.

Code:

```

USE college_db;

-- Basic SELECT
SELECT * FROM students;
SELECT name, email, marks FROM students;

-- WHERE clause
SELECT name, marks FROM students
WHERE marks >= 75;

-- Multiple conditions
SELECT * FROM students
WHERE branch = 'IT' AND marks > 80;

```

```

SELECT * FROM students
WHERE branch = 'IT' OR branch = 'CSE';

-- IN operator
SELECT name FROM students
WHERE branch IN ('IT', 'CE', 'CSE');

-- BETWEEN (inclusive)
SELECT name, marks FROM students
WHERE marks BETWEEN 70 AND 90;

-- LIKE (pattern matching)
SELECT * FROM students WHERE name LIKE 'A%'; -- starts with A
SELECT * FROM students WHERE email LIKE '%gtu%'; -- contains gtu
SELECT * FROM students WHERE name LIKE '_ob%'; -- 2nd char 'o'

-- IS NULL / IS NOT NULL
SELECT * FROM students WHERE marks IS NULL;

-- ORDER BY
SELECT name, marks FROM students
ORDER BY marks DESC; -- highest first

SELECT name, branch, marks FROM students
ORDER BY branch ASC, marks DESC; -- sort by 2 cols

-- LIMIT and OFFSET (pagination)
SELECT * FROM students ORDER BY marks DESC LIMIT 3; -- top 3
SELECT * FROM students ORDER BY marks DESC LIMIT 3 OFFSET 3; -- page 2

-- DISTINCT
SELECT DISTINCT branch FROM students; -- unique branches

```

Explanation:

SELECT cols FROM table WHERE cond ORDER BY col LIMIT n: full query structure. **AND/OR/NOT:** combine conditions. **IN (list):** match any in list. **BETWEEN a AND b:** range (inclusive). **LIKE '%pattern%':** % matches any chars, _ matches one char. **IS NULL:** check for null. **ORDER BY col ASC/DESC:** sort. **LIMIT n OFFSET m:** pagination. **DISTINCT:** unique values.

Output: Queries filter students by marks, branch, name pattern. Top 3 by marks returned. DISTINCT shows unique branch names. Paginated results with OFFSET.

Practical 4: Aggregate Functions & GROUP BY

Aim: Use aggregate functions (COUNT, SUM, AVG, MAX, MIN) with GROUP BY and HAVING.

Code:

```

USE college_db;

-- Aggregate functions
SELECT COUNT(*) AS total_students FROM students;
SELECT COUNT(DISTINCT branch) AS branch_count FROM students;
SELECT AVG(marks) AS avg_marks FROM students;
SELECT MAX(marks) AS top_marks FROM students;
SELECT MIN(marks) AS low_marks FROM students;
SELECT SUM(marks) AS sum_marks FROM students;

-- GROUP BY: stats per branch
SELECT
    branch,
    COUNT(*) AS students,
    ROUND(AVG(marks),2) AS avg_marks,
    MAX(marks) AS highest,
    MIN(marks) AS lowest
FROM students
GROUP BY branch;

-- HAVING: filter groups (like WHERE but for groups)
SELECT branch, AVG(marks) AS avg_marks

```

```

FROM students
GROUP BY branch
HAVING AVG(marks) > 80;

-- GROUP BY with JOIN
SELECT
    s.branch,
    COUNT(p.placement_id) AS placed_count,
    ROUND(AVG(p.package_lpa),2) AS avg_package
FROM students s
LEFT JOIN placements p ON s.student_id = p.student_id
GROUP BY s.branch
ORDER BY placed_count DESC;

-- WITH ROLLUP (adds grand total row)
SELECT branch, COUNT(*) AS cnt, AVG(marks) AS avg
FROM students
GROUP BY branch WITH ROLLUP;

```

Explanation:

COUNT(*): count all rows. **COUNT(col)**: count non-null. **AVG/SUM/MAX/MIN**: math aggregates. **GROUP BY col**: one result row per unique value of col. **HAVING condition**: filter after grouping (WHERE filters before). Non-aggregated columns in SELECT must appear in GROUP BY. **ROUND(val,n)**: round to n decimals. **WITH ROLLUP**: adds subtotal/grand total rows.

Output: COUNT shows 5 total. GROUP BY shows per-branch stats. HAVING filters to branches with avg > 80. JOIN aggregate shows placed count and avg package per branch.

Practical 5: JOINS & Subqueries

Aim: Combine multiple tables with **INNER, LEFT, RIGHT JOIN** and use subqueries.

Code:

```

USE college_db;

-- INNER JOIN: only matching rows
SELECT s.name, s.branch, p.company, p.package_lpa
FROM students s
INNER JOIN placements p ON s.student_id = p.student_id;

-- LEFT JOIN: all students, placement or NULL
SELECT
    s.name,
    s.branch,
    COALESCE(p.company, 'Not Placed') AS status,
    p.package_lpa
FROM students s
LEFT JOIN placements p ON s.student_id = p.student_id;

-- RIGHT JOIN: all placements, student or NULL
SELECT s.name, p.company, p.package_lpa
FROM students s
RIGHT JOIN placements p ON s.student_id = p.student_id;

-- Subquery in WHERE
SELECT name, marks FROM students
WHERE marks > (SELECT AVG(marks) FROM students);

-- Subquery in FROM (derived table)
SELECT branch, avg_m
FROM (
    SELECT branch, AVG(marks) AS avg_m
    FROM students
    GROUP BY branch
) AS branch_avg
WHERE avg_m > 80;

-- Subquery with IN
SELECT name FROM students

```

```

WHERE student_id IN (
    SELECT student_id FROM placements
    WHERE package_lpa > 5
);

-- EXISTS
SELECT name FROM students s
WHERE EXISTS (
    SELECT 1 FROM placements p
    WHERE p.student_id = s.student_id
);

-- Self JOIN (find students with same marks)
SELECT a.name, b.name, a.marks
FROM students a
JOIN students b ON a.marks = b.marks
AND a.student_id < b.student_id;

```

Explanation:

INNER JOIN: rows matching in BOTH tables. **LEFT JOIN:** all left rows + matching right (NULL if no match). **RIGHT JOIN:** all right + matching left. **FULL OUTER JOIN:** all rows both sides. **COALESCE(a,b):** returns first non-null. **Subquery:** query inside query in WHERE/FROM/SELECT. **EXISTS:** true if subquery returns any row. Subquery in FROM must be aliased.

Output: INNER JOIN shows only 3 placed students. LEFT JOIN shows all 5 with unplaced as 'Not Placed'. Subquery finds above-average students. EXISTS finds placed students.

-- End of Practicals --